SLM for Quantum Circuit Optimization

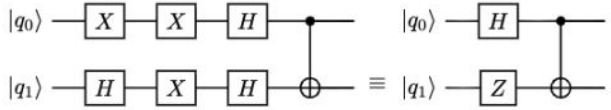
Lukas, Kieran, Anupam

Introduction

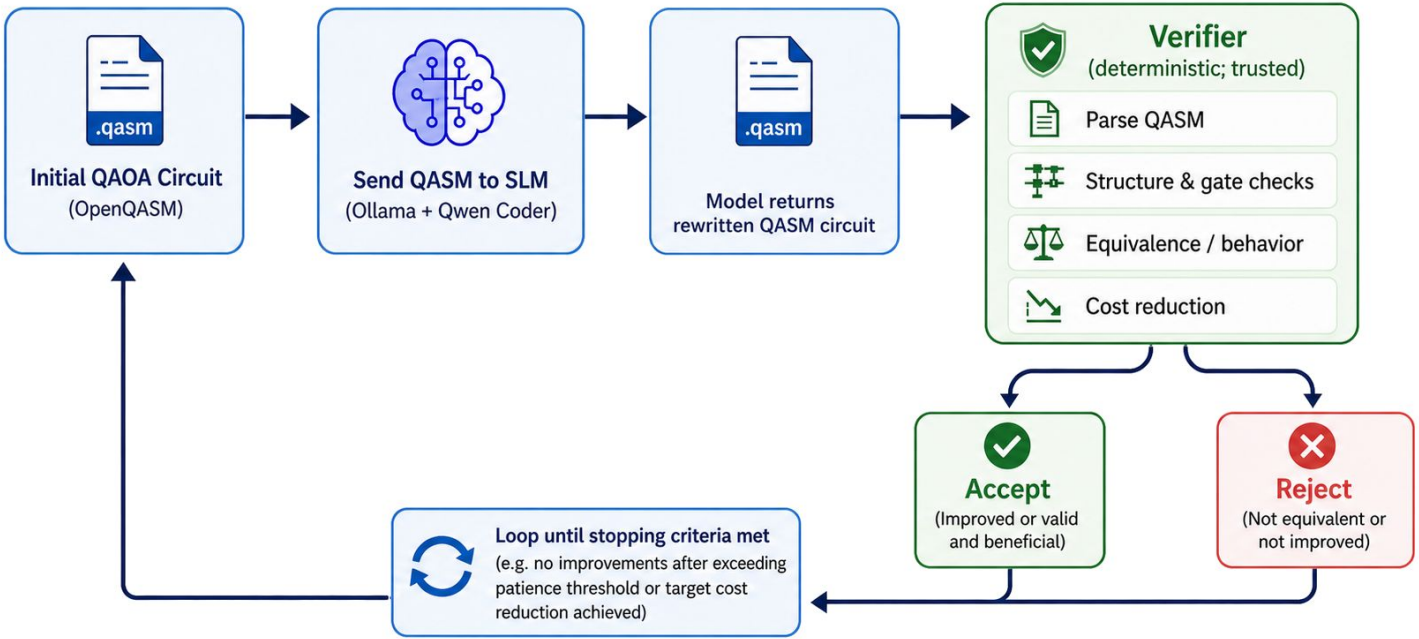
01 Definition & Scale Small Language Models (SLMs) are defined as having from a few million up to 20 billion parameters.

02 Context Limitation They typically struggle to maintain and infer deeper meaning from longer contextual sequences.

Application Context **Quantum Circuit Optimization**



Architecture: Overview



Architecture: Optimizer Loop

The optimizer loop is a verifier-mediated greedy search.

The loop constantly iterates over the circuit, requesting from the model a new optimization over the previous iteration.

The loop **stop** conditions:

- Current circuit cost <= Target cost (i.e. a 10% cost reduction over the initial circuit cost)
- No verified improvements for N consecutive proposals (N is what we call the “patience threshold”)
- Max number of iterations reached
- Max number of model calls

Architecture: Verifier

We all know models love to make stuff up; the verifier is our deterministic **trust** module.

Key responsibilities:

- Validate the proposed circuit is syntactically correct (e.g. it doesn't just invent gates).
- Exact unitary equivalence for smaller circuits or behavioural checks for parameterized QAOA circuits
- Estimate the cost and does it decrease over the previous iteration. Here we used a (fairly crude*) formula using arbitrary weights:

$$C = \alpha D + \beta N_{2q} + \gamma N_{SWAP}$$

$$D = \text{depth count}, \quad N_{2q} = \text{num of two-qubit gates}, \quad N_{SWAP} = \text{num of SWAP gates}$$

* Not a proof that the circuit is physically optimal - **not** hardware-aware!

Open-weight Models: Version 1 - Full QASM Rewrites

Filled with optimism and hope, we asked the model to fully optimize a QASM circuit without the optimization loop.

➡️ **Input**:

- Complete QASM circuit.

⬅️ **Output**:

- Complete rewritten (potentially optimized) QASM circuit.

🤗 **The Good**:

- Recognized simple optimizations like adjacent inverse pairs

🫣 **The Bad**:

- Major hallucinations when any complexity was added to the circuit.

Open-weight Models: Version 2 - Iterative Optimizations

After the disappointment of phase 1, we rolled up our sleeves and tightened up the constraints.

➡️ **Input**:

- Compact instruction-level summary that included gate index, operation, qubit index and gate-specific parameters - no QASM.
- Explicitly asked to do only **ONE** optimization.

⬅️ **Output**:

- Formalised proposal that we could programmatically apply to a circuit.

🤗 **The Good**:

- Could reliably handle complex circuits (up to number 6 of the examples in the repo).
- It didn't take long to propose an optimization.

🫣 **What went wrong**:

- The system prompt was lengthy and had to be uncomfortably explicit.
- This method required far more compiler-like coding as the input/outputs were custom data structures, so custom parsers had to be created to convert to useful circuits.

Open-weight Models: Key Takeaways

Takeaway 01

Fast and efficient

The relatively small model size (3-7b) ran easily run on a laptop CPU without any GPU requirement.

Takeaway 02

Simple local rewrites

Achieved relatively simple optimizations (e.g. redundant inverse pairs or rotation merges), but took a lot of tweaking*.

Takeaway 03

Don’t overwhelm it

Performed much better if treated as a constrained proposal generator rather than a circuit generator.

Takeaway 04

Not trustful

Deterministic verification is absolutely necessary - the model cannot be trusted to grade its own homework!

Takeaway 05

Be explicit

An extensive amount of system prompt engineering with uncomfortably explicit instructions.

Takeaway 06

Not hardware-aware!

The cost function used abstract circuit metrics and did not yet account for a specific hardware.

Fine Tuned SLMs: Experiments and Results

Synthetic dataset generation:

Types:

- Big to small
- Small to small
- Big to big
- Structured - Well known algorithms
- Identity - Empty
- No change

Construct both a noisy and clean circuit.

Noisy constructed by inserting Identity gates: XX, YY, ZZ, CNOT_CNOT.

Fine Tuned SLMs: Experiments and Results

MODEL_NAME = "Qwen/Qwen3.5-4B"

Fine-tuned using QLoRA(Quantized Low-Rank Adaptation).

Shown with https://editor.codecogs.com/, ignore resolution.

$$W \in \mathbf{R}^{4096 \times 4096}$$
$$W' \in \mathbf{R}^{16 \times 4096}$$

Ideally going from 16 bit to 4 bit means 4/16 = 1/4 reduction in memory.

Rule of thumb: 16GB of GPU memory per 1B parameters in the model(Some random person - https://modal.com/blog/how-much-vram-need-fine-tuning)

For 4B around 64 GB VRAM, with QLoRA it is 16 GB

Fine Tuned SLMs: Experiments and Results

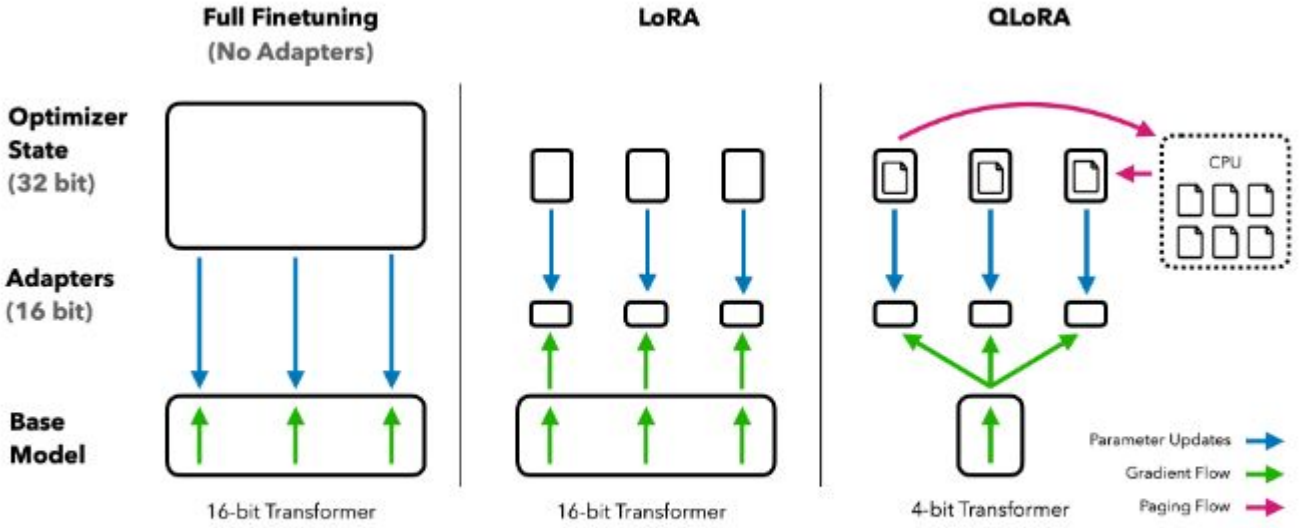


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

arXiv:2305.14314 [cs.LG]

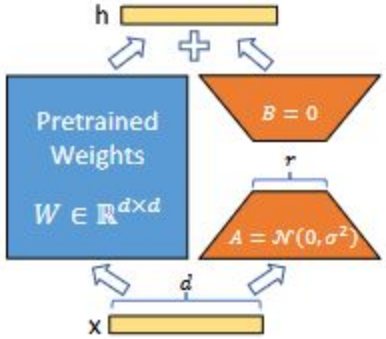
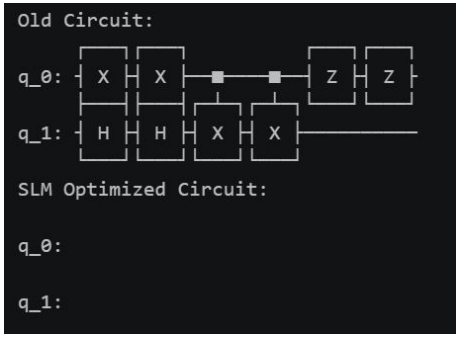
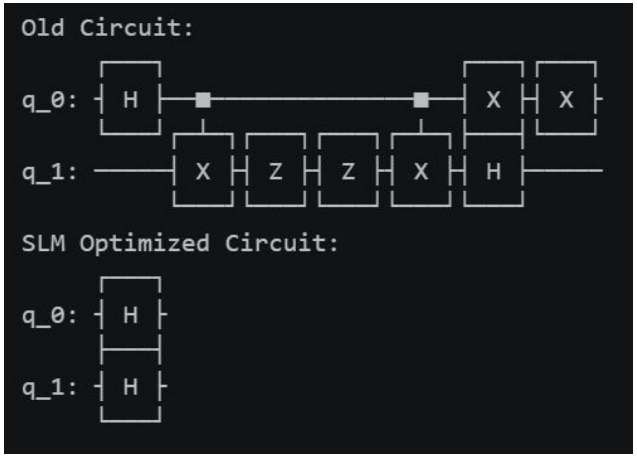


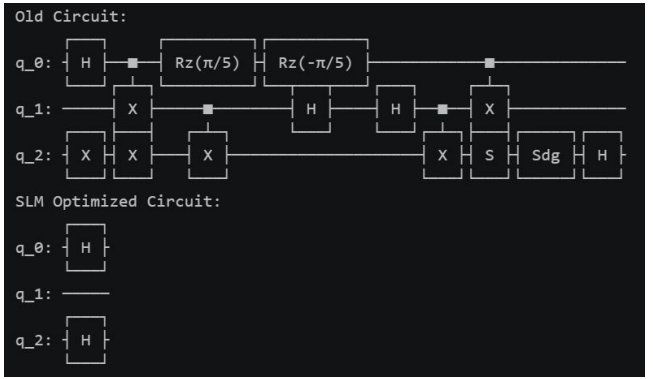
Figure 1: Our reparametrization. We only train A and B.

arXiv:2106.09685 [cs.CL]

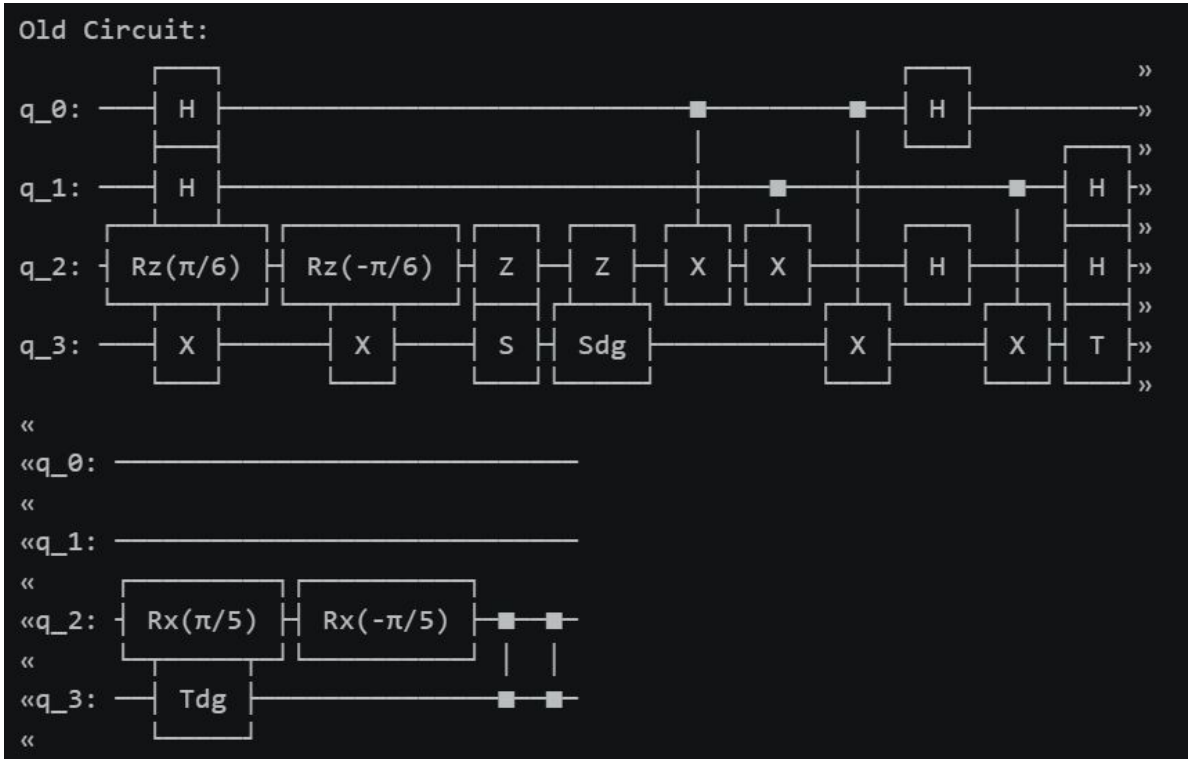
Fine Tuned SLMs: Experiments and Results



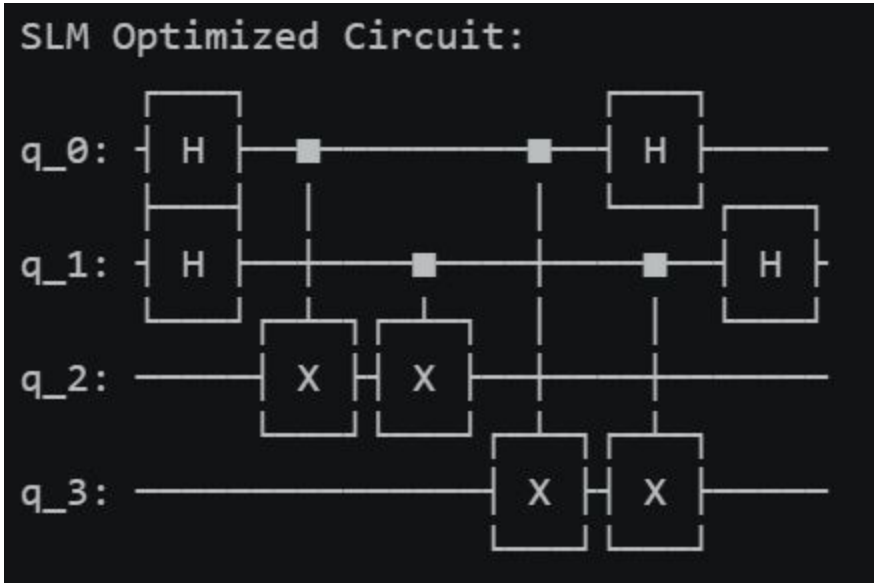




Fine Tuned SLMs: Experiments and Results



Fine Tuned SLMs: Experiments and Results



Quantum Circuit as Directed Acyclic Graph

Observations on SLMs:

Capabilities

What they do well:

Strong reasoning when prompted effectively, but limited in generating optimized circuit code.

Limitations

Key bottlenecks:

Struggle with long-form code retention and line-by-line accuracy.

Quantum Circuit as Directed Acyclic Graph

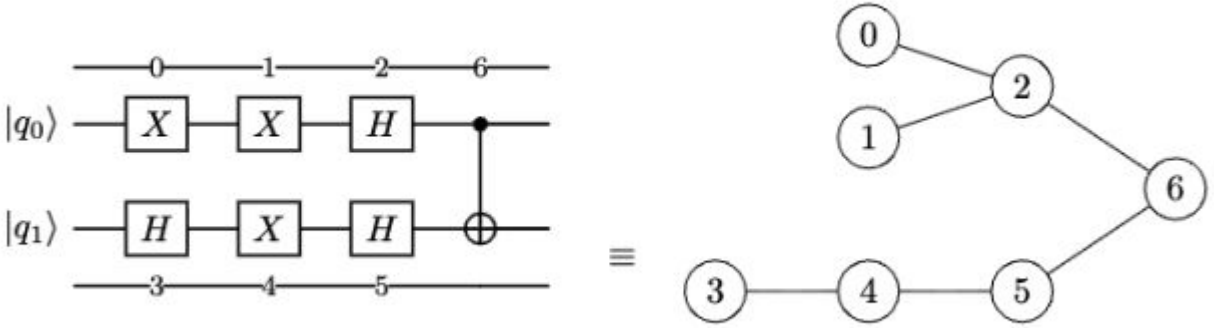
Exact and practical pattern matching for quantum circuit optimization

Raban Iten[*†1,2], Romain Moyard[*‡1], Tony Metger[§1], David Sutter[¶1,2], and Stefan Woerner[‖2]

[1]Institute for Theoretical Physics, ETH Zurich, Switzerland
[2]IBM Quantum, IBM Research – Zurich, Switzerland

Abstract

Quantum computations are typically compiled into a circuit of basic quantum gates. Just like for classical circuits, a quantum compiler should optimize the quantum circuit, e.g. by minimizing the number of required gates. Optimizing quantum circuits is not only relevant for improving the runtime of quantum algorithms in the long term, but is also particularly important for near-term quantum devices that can only implement a small number of quantum gates before noise renders the computation useless. An important building block for many quantum circuit optimization techniques is pattern matching, where given a large and a small quantum circuit, we are interested in finding all maximal matches of the small circuit, called pattern, in the large circuit, considering pairwise commutation of quantum gates.




```
=== QUANTUM CIRCUIT CANONICAL GRAPH ===
GATES (NODES):
 - Node 0: X on qubit(s) [0]
 - Node 1: X on qubit(s) [0]
 - Node 2: H on qubit(s) [0]
 - Node 3: H on qubit(s) [1]
 - Node 4: X on qubit(s) [1]
 - Node 5: H on qubit(s) [1]
 - Node 6: CX on qubit(s) [0, 1]

NON-COMMUTING DEPENDENCIES (DIRECTED EDGES):
 - Node 1 -> Node 2
 - Node 0 -> Node 2
 - Node 3 -> Node 4
 - Node 4 -> Node 5
 - Node 5 -> Node 6
 - Node 2 -> Node 6
```

```
{
  "nodes_to_delete": [3],
  "reason": "Node 0 (X q[0]) commutes with Node 1 (X q[1]). Since they are
together, they are cancelling each other.",
}
```

```
  {
    "reason": "Node 0 (X q[0]) commutes with Node 1 (X q[1]). Since they are
  together, they are cancelling each other.",
    "nodes_to_delete": [0, 1],
  }
```

Addressing Scale in Circuit Optimization

The Challenge

Context Limitation

While an improvement over direct input, this method struggles with large circuits because small language models lose context over extended sequences.

Approach 01

Targeted Analysis

Evaluate smaller circuit sections (sub-graphs) using specific, focused prompts (following the paper's advice).

Approach 02

Integration

Programmatically combine these optimized subgraph back into the complete circuit.

Demonstration Time!

Question Time ⁉️